

CSE406 Assignment Concept Notes

CSE 406 Assignment 01 — Concept Notes

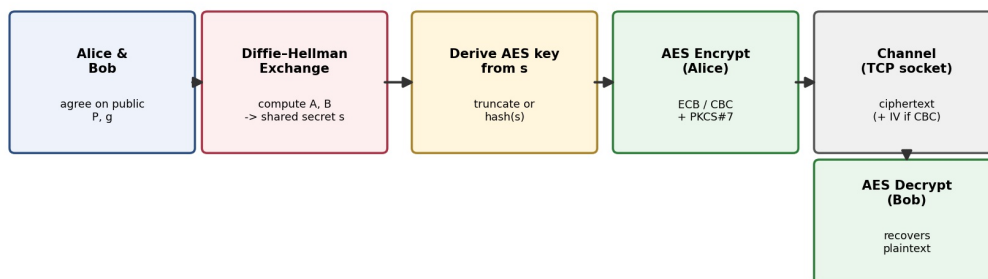
1. The Big Picture: What Are You Actually Building?
2. Diffie-Hellman Key Exchange (Task 2)
3. AES — Advanced Encryption Standard (Task 1)
4. Task 3 — Wiring It Together With Sockets
5. Bonus Tasks Explained
6. Quick Glossary
7. Suggested Order of Implementation

CSE 406 Assignment 01 — Concept Notes

AES + Diffie-Hellman Cryptosystem: Everything Explained

This note walks through every term, structure, and process mentioned in the assignment so you understand *why* each step exists before you write code.

Overall Cryptosystem Pipeline



Overall cryptosystem pipeline

1. The Big Picture: What Are You Actually Building?

Two parties, **Alice** (sender) and **Bob** (receiver), want to communicate secretly over a channel anyone can eavesdrop on. The system has two halves:

1. **Diffie-Hellman (DH) Key Exchange** — lets Alice and Bob agree on a *shared secret number* without ever sending that number itself over the channel.
2. **AES (Advanced Encryption Standard)** — uses that shared secret as a symmetric key to actually scramble (encrypt) and unscramble (decrypt) the message.

So DH solves “how do two strangers agree on a secret key in public?” and AES solves “given a secret key, how do I encrypt data fast and securely?” You need both because AES alone requires the key to already be shared somehow — DH is how you share it.

2. Diffie-Hellman Key Exchange (Task 2)

2.1 The problem it solves

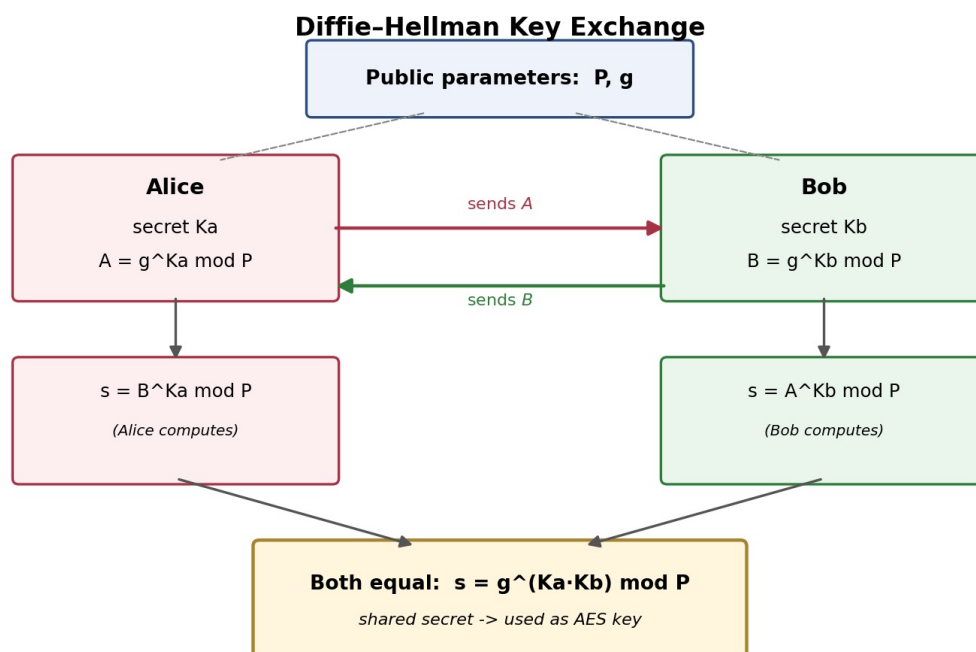
If Alice just picked a key and emailed it to Bob, anyone reading the email would have the key too. DH lets them compute a *shared* number using **public** information plus a **private secret each keeps to themselves**, such that an eavesdropper who sees everything public still can't feasibly compute the shared number.

2.2 The math objects

Symbol	Meaning
P	A large prime number — the modulus. Public.
g	A generator/base — public. $1 < g < P$.
K_a	Alice's secret random number (private, never sent).
K_b	Bob's secret random number (private, never sent).
$A = g^{K_a} \bmod P$	Alice's public value, sent to Bob.
$B = g^{K_b} \bmod P$	Bob's public value, sent to Alice.
s	The shared secret: $s = g^{(K_a \cdot K_b)} \bmod P$, computed by both sides independently.

2.3 Why it's secure: the Discrete Logarithm Problem

Given g , P , and $A = g^{K_a} \bmod P$, it is computationally hard to recover K_a . This is the **discrete logarithm problem** — the “one-way function” that DH security rests on. Squaring/exponentiating forward (compute $g^{K_a} \bmod P$) is fast; going backward (recover K_a from A) is infeasible for large enough P . An eavesdropper sees P , g , A , B but not K_a or K_b , so they can't compute s either.



Diffie-Hellman key exchange

2.4 Why both sides get the same s

Alice computes: $s = B^{K_a} \bmod P = (g^{K_b})^{K_a} \bmod P = g^{(K_b \cdot K_a)} \bmod P$
Bob computes: $s = A^{K_b} \bmod P = (g^{K_a})^{K_b} \bmod P = g^{(K_a \cdot K_b)} \bmod P$

Since exponents commute ($K_a \cdot K_b = K_b \cdot K_a$), both land on the same number s . This is the entire trick of DH.

2.5 Modular exponentiation

You need to compute things like $g^{K_a} \bmod P$ where K_a can be a 128+ bit number — way too large to compute g^{K_a} first and then mod it (the intermediate number would have millions of digits). **Fast modular exponentiation** (a.k.a. “square-and-multiply” or “exponentiation by squaring”) computes this in $O(\log(\text{exponent}))$ multiplications, each followed immediately by a mod, keeping numbers small throughout. Python’s built-in `pow(base, exp, mod)` already does this efficiently — you’re explicitly allowed and encouraged to use it instead of writing your own.

2.6 Generating the prime P

- You need a **random prime number** that is exactly k bits long ($k = 128, 192, \text{ or } 256$ — matching your AES key size).
- Process: generate a random odd k -bit number, then **test if it’s prime**. If not, try another. Repeat until you find one.
- **Miller-Rabin primality test**: a *probabilistic* test. It doesn’t prove a number is prime with 100% certainty, but it can prove a number is **composite** (not prime) with certainty, and if a number passes many rounds of the test, the probability it’s actually composite becomes vanishingly small (e.g., less than 1 in 2^{80}). It’s fast, which is why it’s used instead of a slow deterministic test — you can use Python’s `sympy.isprime` or implement Miller-Rabin yourself; both are acceptable (“packages for prime and random generation only”).

2.7 Choosing a valid generator g

Not every number between 1 and P works well as a generator — some produce a small set of possible outputs (a small “subgroup”), which would make A and B easier to guess. A proper generator should produce outputs covering (ideally) the whole group of numbers $\{1, \dots, P-1\}$, maximizing the search space for an attacker.

- The condition given: for every prime factor r of $P - 1$, check $g^{(P-1)/r} \bmod P \neq 1$. If this holds for all prime factors of $P-1$, g is a valid generator of the full group (or a large enough subgroup) modulo P .
- Practically: pick a candidate g (e.g., 2, 3, 5, ...), factor $P - 1$ into its prime factors, test the condition, and if it fails, try the next candidate g .

2.8 Private keys K_a, K_b

Random numbers **at least k bits long**, kept secret. Larger private keys (comparable in size to P) make brute-forcing them harder.

2.9 From shared secret s to AES key

s is just a big number (could be much larger than 128 bits). AES needs a key of exactly 128 (or 192/256) bits. Two reasonable options (you must pick one and *justify it in your report*): - **Truncate**: take the lowest k bits of s . - **Hash then truncate**: run s through a hash function (e.g., SHA-256) and take the first k bits of the digest. This is generally considered better practice because it spreads any structure in s more uniformly across the derived key.

2.10 Reproducibility

“Fixed random seed” means calling something like `random.seed(42)` before generating P, g, K_a, K_b so that re-running your program produces identical output — useful for demonstrating/debugging, though real systems would never do this (they need true randomness).

2.11 Performance table

You’re asked to time, for each key size $k \in \{128, 192, 256\}$, how long it takes to compute A, B , and the

shared secret s , averaged over ≥ 5 runs. Larger $k \rightarrow$ larger numbers $\rightarrow$ slower exponentiation, so you should see times increase with k .

3. AES — Advanced Encryption Standard (Task 1)

AES is a **block cipher**: it encrypts data in fixed-size chunks called **blocks** (128 bits = 16 bytes each), using a secret key, through multiple repeated **rounds** of scrambling operations.

3.1 Key sizes and round counts

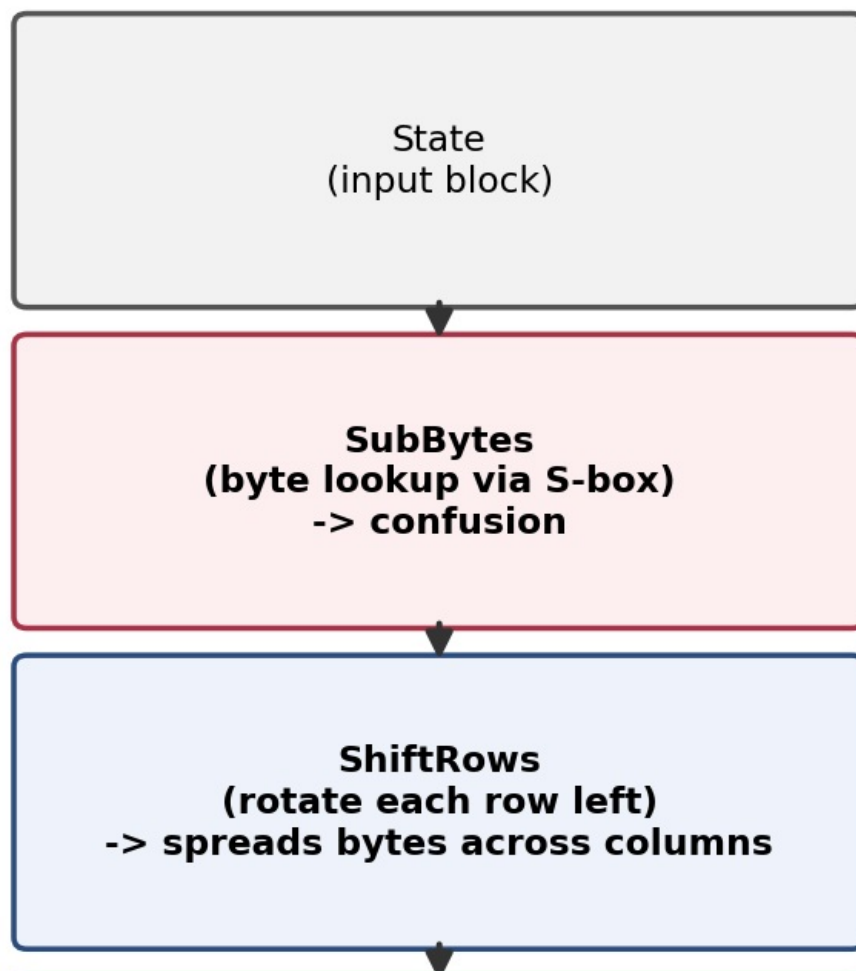
Key size	Rounds
128-bit	10
192-bit	12
256-bit	14

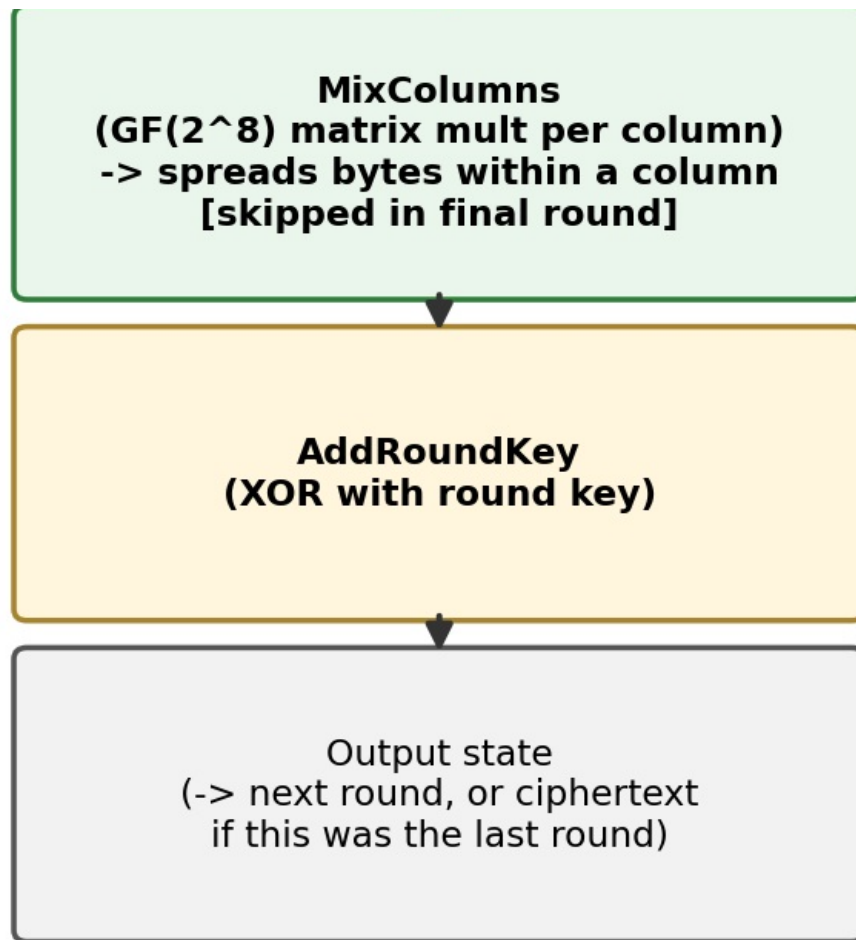
You're implementing AES-128 (10 rounds) as the core requirement; 192/256-bit support is a bonus.

3.2 The “state”

AES treats each 16-byte block as a **4×4 grid of bytes** (called the *state*), filled column by column. All four round operations transform this grid.

One AES Round (encryption direction)





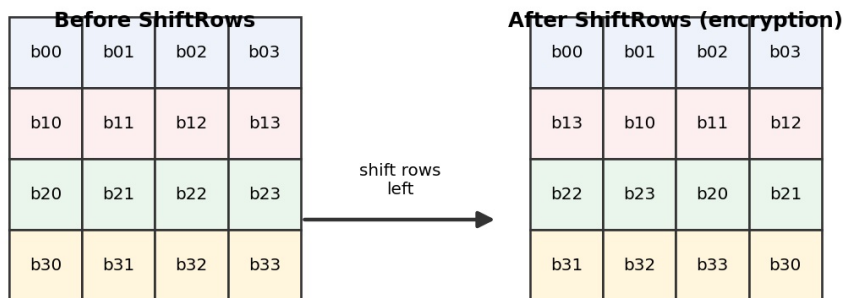
One AES round

3.3 The four round operations

1. SubBytes (byte substitution) Each byte in the state is replaced using a fixed lookup table called the **S-box** (substitution box) — a precomputed 256-entry table (one output for every possible byte value 0-255). This introduces **non-linearity** (confusion), which is essential — without it, AES would be a simple linear transformation, easily broken. Decryption uses the **InvSbox** (inverse S-box) to undo this substitution. Both tables are provided to you in `aes_helpers.py`, so you just do the table lookup, not derive the tables.

2. ShiftRows The 4×4 state has rows `R0`, `R1`, `R2`, `R3`. Each row is cyclically shifted left (for encryption): - `R0` shifted by 0 (unchanged) - `R1` shifted by 1 byte - `R2` shifted by 2 bytes - `R3` shifted by 3 bytes

This spreads byte values across columns (**diffusion**) so that a change in one byte eventually affects the whole block over multiple rounds. Decryption shifts rows **right** by the same amounts to undo it.



Row 0: shift left 0 Row 1: shift left 1
 Row 2: shift left 2 Row 3: shift left 3
 (Decryption shifts right by the same amounts)

ShiftRows illustration

3. MixColumns Each column of the state (4 bytes) is multiplied by a fixed 4×4 matrix (`Mixer`) using **Galois Field $GF(2^8)$ arithmetic** — not ordinary integer arithmetic. This further mixes bytes *within* a column, combined with ShiftRows' cross-row mixing, so every output byte in the next round depends on multiple input bytes (full diffusion). Decryption uses the inverse matrix (`InvMixer`). - You don't need to implement $GF(2^8)$ math yourself — `gf_mult(a, b)` (Galois field multiplication) is provided. - **Important:** the final (10th) round *skips* MixColumns — this is a standard part of the AES specification, not a mistake.

4. AddRoundKey The state is XORed byte-by-byte with a 128-bit **round key** — a different 128-bit key derived for each round from the original key via the **key schedule / key expansion**. XOR is its own inverse, so this same step is used identically in both encryption and decryption.

3.4 Key Schedule (Key Expansion)

The original 128-bit key must be expanded into **11 round keys** (1 for the initial AddRoundKey before round 1, plus 1 per round $\times$ 10 rounds), each 128 bits, using: - Byte substitution (via S-box) - Word rotation - XOR with **Rcon** (round constants) — a small fixed table of constants, one per round, that prevents the key schedule from having symmetric/repeating patterns (provided to you). - XOR with previous words

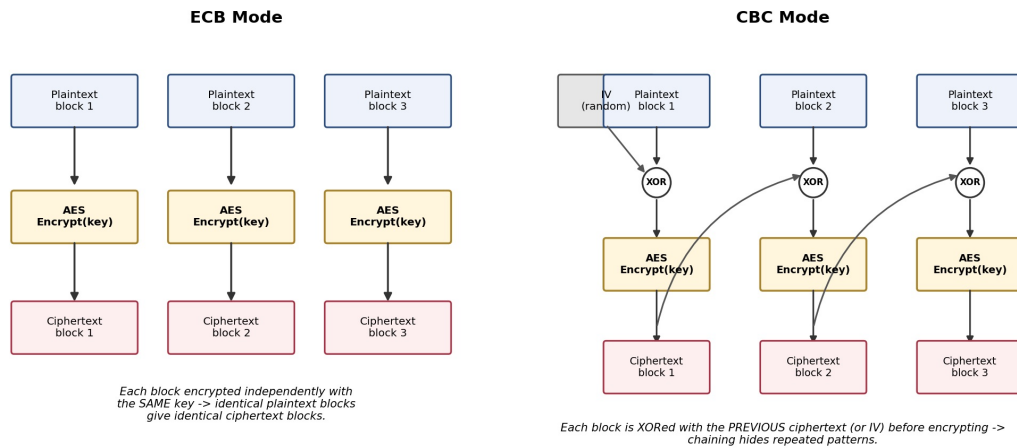
This is the standard **Rijndael key schedule**. You must implement this logic yourself (only the S-box/Rcon tables are provided, not the schedule algorithm).

3.5 Encryption vs Decryption structure

- **Encryption:** AddRoundKey (initial) $\rightarrow$ [SubBytes $\rightarrow$ ShiftRows $\rightarrow$ MixColumns $\rightarrow$ AddRoundKey] $\times$ 9 $\rightarrow$ [SubBytes $\rightarrow$ ShiftRows $\rightarrow$ AddRoundKey] (final round, no MixColumns).
- **Decryption:** mirrors this in reverse using **InvSubBytes**, **InvShiftRows**, **InvMixColumns**, applying round keys in reverse order.

3.6 Modes of Operation — why a single-block cipher needs a “mode”

AES by itself only encrypts one 16-byte block. Real messages are longer than 16 bytes, so you need a **mode of operation**: a recipe for chaining multiple block encryptions together.



ECB vs CBC modes

ECB (Electronic Codebook) - Split plaintext into 16-byte blocks; encrypt **each block independently** with the same key. - Weakness: identical plaintext blocks always produce identical ciphertext blocks — patterns in the input (like the flat-color regions of an image) remain visible in the output. This is exactly what the bonus ECB-vs-CBC image task demonstrates. - No IV needed (there's nothing to "chain").

CBC (Cipher Block Chaining) - Before encrypting each plaintext block, **XOR it with the previous ciphertext block** first, then encrypt. - The very first block has no "previous ciphertext," so a random **Initialization Vector (IV)** — a random 16-byte block — is XORed with it instead. - Because each block's encryption depends on all preceding ciphertext, identical plaintext blocks produce *different* ciphertext (as long as what precedes them differs), hiding patterns — hence CBC ciphertext of an image looks like noise. - The IV isn't secret, but must be random and (ideally) unique per encryption. Per the assignment: prepend the IV as the first 16 bytes of your output, followed by the actual ciphertext, so the receiver can extract it before decrypting.

PKCS#7 Padding AES needs plaintext that's an exact multiple of 16 bytes. If your final block is short, PKCS#7 padding fills the missing bytes with a value **equal to the number of padding bytes added**. E.g., if 5 bytes of padding are needed, append `0x05 0x05 0x05 0x05 0x05`. If the plaintext is already a multiple of 16 bytes, a **full extra block of padding** (16 bytes of `0x10`) is still added — this is required so that decryption can always unambiguously identify and strip padding (by reading the last byte's value and removing that many bytes from the end).

3.7 What "independent implementation" means here

You must write `subBytes`, `shiftRows`, `mixColumns` (calling the provided `gf_mult`), `addRoundKey`, the key schedule, ECB, CBC, and padding **yourself**. Only the following are given (in `aes_helpers.py`) because they're fixed lookup tables/data, not algorithms: - `Sbox`, `InvSbox` (substitution tables) - `Rcon` (round constants for key schedule) - `Mixer`, `InvMixer` (matrices for `MixColumns`/`InvMixColumns`) - `gf_mult(a, b)` ($GF(2^8)$ multiplication helper)

3.8 Output requirements

For each mode, your program should clearly display: which mode (ECB/CBC), the key, the original plaintext, the ciphertext, and the recovered (decrypted) plaintext — each shown in both **ASCII** and **hexadecimal** representations, plus timing for key-schedule, encryption, and decryption.

4. Task 3 — Wiring It Together With Sockets

TCP sockets let two separate running programs (potentially on different machines) send bytes back and forth over a network connection — this simulates Alice and Bob being physically separate.

Sequence: 1. Alice generates `P`, `g`, her secret `Ka`, computes $A = g^{Ka} \bmod P$, and sends `(P, g, A)` to Bob. 2.

Bob picks his secret K_b , computes $B = g^{K_b} \bmod P$, and sends B back to Alice. 3. Both independently compute the shared secret s (Alice: $s = B^{K_a} \bmod P$; Bob: $s = A^{K_b} \bmod P$) and derive the AES key from it. 4. Alice AES-encrypts her plaintext using the shared key and sends the ciphertext (with IV, if CBC) over the socket. 5. Bob receives the ciphertext, decrypts it with the same derived key, and recovers the plaintext.

This demonstrates the full pipeline from the diagram in the assignment: $DH \rightarrow$ shared key $\rightarrow$ AES encrypt $\rightarrow$ channel $\rightarrow$ AES decrypt.

5. Bonus Tasks Explained

- ECB vs CBC on an image:** Encrypt a small image's raw pixel bytes with both ECB and CBC (keep the file header intact so the image is still viewable/openable). ECB will visibly retain the image's shapes because identical-colored regions (identical plaintext blocks) map to identical ciphertext blocks. CBC will look like random static because chaining breaks that block-to-block repetition.
- Other file types:** Generalize your AES encrypt/decrypt functions to work on raw bytes from any file (not just text strings), and send such files over your Task 3 socket connection.
- Larger keys:** Extend your key schedule and round structure to correctly handle 192-bit (12 rounds) and 256-bit (14 rounds) AES keys, per the table in Section 3.1.

6. Quick Glossary

Term	One-line meaning
Symmetric key	Same key used for both encryption and decryption.
Block cipher	Encrypts fixed-size chunks (blocks) of data at a time.
State	The 4×4 byte grid AES operates on internally.
S-box / InvSbox	Fixed substitution lookup table (and its inverse) for byte confusion.
Rcon	Round constants used only in the key schedule.
GF(2⁸)	The finite field AES's MixColumns arithmetic is defined over.
Confusion	Making the relationship between key and ciphertext complex (via SubBytes).
Diffusion	Spreading influence of one plaintext/key bit across many ciphertext bits (via ShiftRows/MixColumns).
Key schedule / key expansion	Algorithm deriving all round keys from the original key.
ECB	Encrypt each block independently — patterns leak.
CBC	Chain blocks via XOR with previous ciphertext — patterns hidden.
IV (Initialization Vector)	Random block XORed with the first plaintext block in CBC.
PKCS#7 padding	Padding scheme where each pad byte's value equals the pad length.
Discrete logarithm problem	Hardness of recovering exponent x from $g^x \bmod P$.
Generator (primitive root)	A base g whose powers mod P span the full group.

Miller-Rabin test	Fast probabilistic primality test.
Modular exponentiation	Efficiently computing $\text{base}^{\text{exp}} \bmod P$ without huge intermediate numbers.
TCP socket	Network connection abstraction for sending bytes between two programs.

7. Suggested Order of Implementation

1. Get `aes_helpers.py` imported correctly (note the numeric-prefix filename issue mentioned in submission guidelines — use `importlib` if your own filename starts with digits).
2. Implement key schedule first, test it against known AES-128 test vectors if possible.
3. Implement single-block encrypt/decrypt (Sub/Shift/Mix/AddRoundKey + inverses).
4. Wrap in ECB mode with padding; verify encrypt→decrypt round-trip recovers original text.
5. Add CBC mode with random IV; verify round-trip.
6. Add timing instrumentation.
7. Implement Miller-Rabin (or use `sympy.isprime`) + prime generation for DH.
8. Implement generator selection, key generation, shared secret computation, and timing table.
9. Combine into a socket-based Alice/Bob demo (Task 3).
10. Only after core tasks work, attempt bonus tasks.

Good luck — and remember: no code copied from the web, per the plagiarism policy in the assignment.